How much do standard evaluations overstate ML intrusion detection? A leakage-aware benchmark on UNSW-NB15

[Your Name] · code, data checksums and all results: see the accompanying repository

Abstract. Published intrusion-detection results are often measured on splits that leak near-identical flows between training and test data. I audit the official UNSW-NB15 partitions, clean them, and compare three baselines (logistic regression, random forest, gradient boosting) under three protocols: the official split, a random split of the cleaned data, and a split that keeps clusters of near-duplicate flows on one side. Moving from the random to the grouped protocol lowers the best model's macro-F1 from 0.70 to 0.43 and raises its false-positive rate at 95% detection from 0.12 to 0.18. Many remaining errors come from contradictory labels in the data rather than from the models.

1. Data and problems found

I used the official training (175,341 flows) and testing (82,332) CSVs, pinned by SHA-256 (obtained from GitHub mirrors, since UNSW's server was unreachable; the training file was byte-identical across two mirrors). Three problems matter. **Duplicates:** 42% of training and 34% of test rows are exact feature duplicates. **Leakage:** 8,541 test rows (10.4%) have an exact twin in the training set and 20.4% a near-twin (nearest-neighbour distance < 0.1 in standardised log-feature space). **Contradictory labels:** 414 feature vectors carry both benign and malicious labels (dropped) and 2,031 more carry several attack categories (kept once, majority category); 77% of Analysis, 85% of Backdoor and 77% of DoS rows are in such vectors. Pooling and cleaning gives 153,270 flows. The class balance changes sharply (Generic: 58,871 raw rows, 7,284 clean), which is why raw-split scores are not comparable with cleaned-data scores.

2. Method

Protocols. P0: the official split as published. P1: stratified random 5-fold on the cleaned pool. P2: stratified *group* 5-fold, where flows sharing protocol, service, state, both TTLs and a coarse log-scale packet/byte profile form one group (no labels used). The official files have no timestamps or IP addresses, so a true time- or session-based split is impossible; P2 is a near-duplicate proxy and a deliberately harsh test. It reduces near-twin leakage from 15.7% (P1) to 0.7% with no shared groups (verified by unit tests). **Models and metrics.** Hyper-parameters were chosen on an inner validation split of one training fold, never on test data. I report per-class precision/recall, macro-F1, binary AUC, and the false-positive rate (FPR) at 95% and 99% detection; the 95% threshold is also chosen on validation data only and then measured on test.

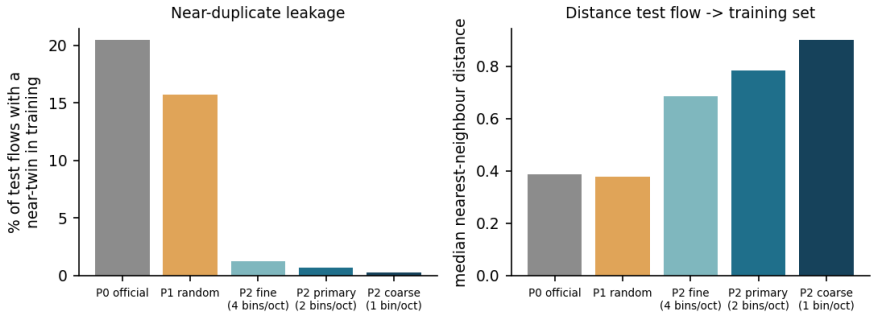


Figure 1. Share of test flows with a near-twin in training (left) and median nearest-neighbour distance (right). Grouping removes almost all near-duplicate leakage.

3. Results

Protocol	Logistic reg.	Random forest	Grad. boosting
P0 official split (raw)	0.37 / 0.23	0.47 / 0.13	0.51 / 0.13
P1 random, deduplicated	0.48 / 0.19	0.66 / 0.13	0.70 / 0.12
P2 grouped (leakage-safe)	0.26 / 0.37	0.38 / 0.22	0.43 / 0.18

Table 1. Macro-F1 / FPR at 95% detection (pooled out-of-fold predictions, seed 42). Fold-to-fold macro-F1 sd under P2 is about 0.025.

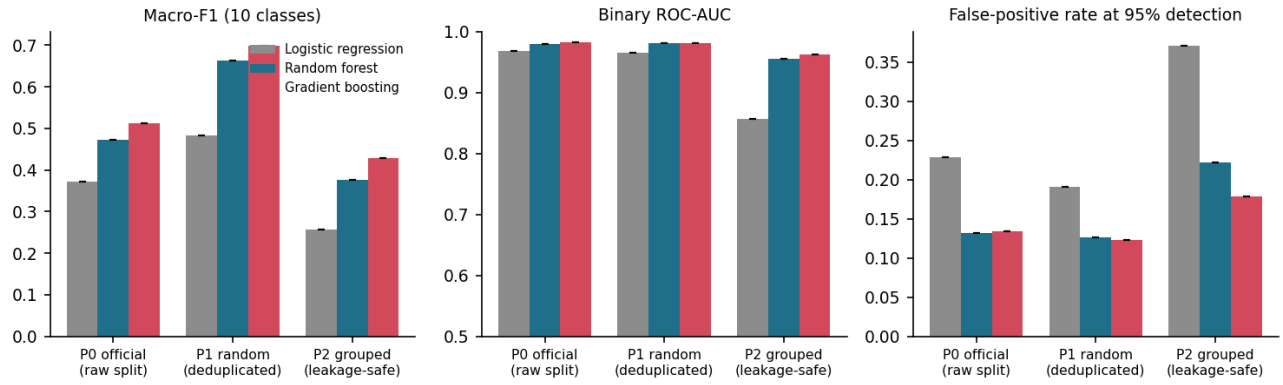


Figure 2. The leakage-safe protocol (P2) is lowest for every model. The official split (P0) is not simply inflated relative to P1: it also carries its own distribution shift. Model ranking is stable across protocols.

With the transferred threshold, gradient boosting under P2 detects 95.3% of attacks at a 19.4% false-positive rate, too noisy to deploy alone. Per class (gradient boosting, P2), Normal (recall 0.90) and Exploits (0.91) are strong, while Generic (0.07), Reconnaissance (0.15) and DoS (0.25) collapse, although they score 0.97, 0.81 and 0.08 under P0. Stricter grouping lowers scores monotonically (gradient boosting macro-F1 0.46, 0.43, 0.40 at 4, 2 and 1 size-bins per octave). Removing TTL features from the random forest raises FPR at 95% detection from 0.22 to 0.37; removing TCP sequence numbers or protocol/service/state changes it little.

4. Error analysis

81% of Generic is predicted as Fuzzers and 64% of DoS as Exploits; the latter matches the label-conflict finding, so part of this error is a data ceiling for flow-level features. At 95% detection nearly all missed attacks are Fuzzers (15.8% missed; every other class under 3%). False alarms concentrate on benign flows with no identified service (74% of all false alarms; 37% of benign HTTP flows are flagged versus 0.2% of DNS). A spurious correlation is visible directly: 93% of UDP/DNS flows in the official training file are attacks, and logistic regression, when a 10,255-flow benign DNS cluster was held out, labelled 10,250 of those flows Exploits. This is the spurious-correlation pitfall described by Arp et al. (2022).

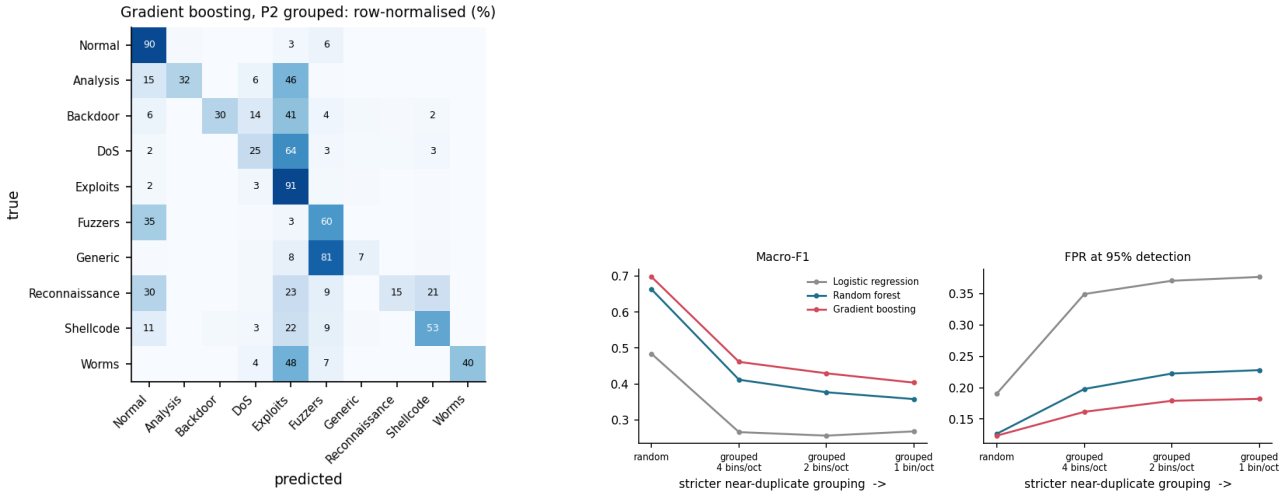


Figure 3. Left: gradient boosting confusion matrix under P2 (row-normalised %). Right: scores fall as near-duplicate grouping gets stricter (sensitivity analysis).

5. Limitations

No true temporal or session split was possible with these files; grouping is a heuristic proxy. All P2 numbers use one fold-assignment seed (fold-level spread is reported instead; AUC ranges 0.59-0.96 for logistic regression). Majority-category resolution of conflicting vectors shrinks rare classes. Baselines are deliberately simple and lightly tuned. Next step: repeat with CIC-IDS2017 and a genuine day/time split.

References. *Moustafa & Slay (2015), UNSW-NB15, MilCIS. Arp et al. (2022), Dos and Don'ts of ML in Computer Security, USENIX Security. Engelen et al. (2021), Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study, IEEE S&P Workshops. Zoghi & Serpen (2021), arXiv:2101.05067.*